PHASE 3 FINAL REPORT

European Weather & Climate Change Analytics Lakehouse

Architecture • Gold Data Model • DAG Analysis • FinOps Evaluation

Item	Details
Scope	Hourly historical weather analytics for nine European cities, 1981–2025.
Source	Open-Meteo Historical Weather API / ERA5 hourly weather.
Platform	Databricks, Delta tables, Apache Spark; Power BI is the intended reporting layer.
Current status	Gold build completed and validation passed in the supplied Databricks output.
Important evidence note	A real Spark UI DAG screenshot and actual workspace billing/DBU details still need to be inserted before final submission.

Evidence integrity: Row counts and runtimes below come from the provided Databricks output. This report does not invent a Spark UI screenshot, actual DBU usage, or official Databricks pricing. A marked placeholder is included for the real DAG capture.

1. Executive Summary

The lakehouse separates ingestion and raw/staged data (Bronze), cleaned hourly weather records (Silver), and analytics-ready Gold tables. The supplied Phase 3 run reports 3,550,176 Silver rows and the same number in fact_weather. Validation reported zero duplicate city/timestamp fact keys, unique dimension keys, zero orphan foreign keys, and unique grains in all aggregate tables.

The initial Gold build and validation succeeded in the supplied output. Automatic Gold refresh after future full and incremental ingestion runs remains a separate integration task and is not marked complete here.

2. Architecture Overview

The architecture above is a logical project diagram, not a screenshot of a live Databricks workspace. Power BI connectivity and dashboard completion should be recorded after the partner completes them.

Weather variables

temperature_2m, relative_humidity_2m, apparent_temperature, precipitation, rain, snowfall, weather_code, pressure_msl, surface_pressure, cloud_cover, wind_speed_10m, and wind_direction_10m.

3. Gold Data Model

The fact table grain is one row per city per hourly timestamp. The intended star schema links the fact table to city and date dimensions.

dim_city PK: city_key city, country, region, latitude, longitude	**fact_weather** FK: city_key FK: date_key timestamp + 12 weather measures source/batch/load metadata	**dim_date** PK: date_key (yyyyMMdd) date/day/month/quarter/year/decade/season

Relationships for Power BI

dim_city[city_key] 1 → * fact_weather[city_key]; dim_date[date_key] 1 → * fact_weather[date_key]. Confirm the actual column names and types in Power BI before creating the relationships.

Table	Purpose	Rows reported
dim_city	One row per city	9
dim_date	One row per calendar date	16,436
fact_weather	Hourly city observations	3,550,176
gold_daily_weather	Daily summaries	147,924
gold_monthly_climate	Monthly summaries	4,860
gold_annual_climate	Annual summaries	405
gold_extreme_weather	Daily project-defined extreme indicators	147,924

Business questions supported

- Annual average temperature trends by city (1981–2025).
- Differences by city, region, season, and decade.
- Frequency of project-defined hot/heavy-rain indicators.
- Regional and seasonal patterns across the historical records.

4. Validation Results

These are the checks reported by Notebook 11 in the supplied output; they have not been independently re-run by this report generator.

Check	Observed result	Status
Silver row count	3,550,176	PASS
Fact row count	3,550,176	PASS
Duplicate fact logical keys	0	PASS
City rows / unique keys	9 / 9	PASS
Date rows / unique keys	16,436 / 16,436	PASS
Orphan city/date keys	0 / 0	PASS
Daily rows / duplicate keys	147,924 / 0	PASS
Monthly rows / duplicate keys	4,860 / 0	PASS
Annual rows / duplicate keys	405 / 0	PASS
Extreme rows / duplicate keys	147,924 / 0	PASS
Timestamp coverage	1981-01-01 00:00 to 2025-12-31 23:00	PASS

5. Spark DAG Analysis

Logical processing flow inferred from the notebook output

1. Read Silver Delta data and count source rows.
2. Build/update city and date dimensions.
3. Merge hourly records into fact_weather using the city/timestamp logical key.
4. Aggregate to daily, monthly, and annual grains.
5. Build daily extreme-weather indicators.
6. Record Gold execution information through the logging helper.

Aggregations and Delta MERGE may cause Spark shuffles and multiple stages. The notebook printout alone does not establish exact stage counts, task counts, shuffle volume, skew, or the slowest Spark stage. Those details must be taken from the actual Spark UI.

REQUIRED: INSERT A REAL SPARK UI DAG SCREENSHOT HERE

Capture the actual DAG/job or stage view from the Databricks run. After inserting the image, add 3–5 observations about stages, shuffles, wide transformations, task distribution, and the slowest relevant stage.

No real Spark UI screenshot was supplied when this report was generated. Do not submit this placeholder as a screenshot.

6. Architecture Decision Log

Area	Decision / rationale	Follow-up
Layering	Bronze → Silver → Gold separates raw ingestion, cleaned data, and analytics.	Keep layer responsibilities documented.
Fact grain	One row per city + hourly timestamp; logical key city_key + timestamp.	Validation reported zero duplicate keys.
Dimensions	City/date dimensions support consistent filtering and reporting.	Validation reported unique dimension keys.
Partitioning	observation_year is intended for time-range filters on first fact creation.	Verify physical Delta partitioning. Do not drop/recreate an existing table without a deliberate migration.
Incremental Gold	Design uses last successful Gold fact-log timestamp as a watermark and merges newer Silver rows.	Test late-arriving data and failed-run handling.
Aggregate refresh	Daily/monthly/annual/extreme aggregates are rebuilt idempotently.	Measure compute cost as data grows.
Schema drift	Unexpected schema changes should be detected/quarantined.	Verify production path, not only controlled test.
PII	Weather observations are environmental data, but logs and metadata still need review.	Check logs and exports for sensitive details.
OPTIMIZE	Optional optimization was skipped in the reported run.	Enable only after measuring query/file performance.

7. FinOps Evaluation

The seven reported table-operation durations sum to 67.45 seconds (about 1.12 minutes). This is a sum of operation timings, not necessarily the entire notebook wall-clock time or billable compute duration.

Metric	Value	Basis
Gold operation-time sum per refresh	67.45 sec	Reported notebook output
Assumed refresh frequency	30 per month	Illustrative assumption
Annual refresh count	360	30 × 12
Annual operation time	6.745 hours	360 × 67.45 sec ÷ 3,600

Illustrative cost sensitivity — these rates are hypothetical, not official Databricks prices

Assumed effective cost	Estimated annual cost	Interpretation
$0.50 / compute-hour	$3.37	Illustrative low scenario
$1.00 / compute-hour	$6.75	Illustrative middle scenario
$2.00 / compute-hour	$13.49	Illustrative high scenario

Formula: annual estimated cost = annual operation-hours × effective compute cost per hour. These scenarios exclude storage, data transfer, warehouse idle time, Bronze/Silver processing, and differences between operation timing and billable runtime. Replace the illustrative rate with the actual effective cost from the workspace billing/usage details.

FinOps recommendations

- Capture actual wall-clock runtime, DBU consumption, compute type, and applicable rate.
- Measure FULL and INCREMENTAL runs separately rather than assuming incremental savings.
- Avoid idle compute and schedule refreshes only when needed.
- Evaluate OPTIMIZE only after checking file sizes and query performance.
- Keep run ID, load type, start/end time, rows processed, and success/error status.

8. Final Submission Checklist

Deliverable	Status	Action before submission
Gold model and aggregate tables	Build and validation passed in supplied output	Keep notebook outputs as evidence.
Power BI dashboard	Not evidenced as completed	Connect to Databricks; set relationships; create at least three visuals plus audit/error monitoring; save .pbix.
Architecture PDF	Draft created	Insert actual Spark UI DAG screenshot and finalize measured FinOps figures.
DAG analysis	Logical flow documented	Add screenshot-based observations from the actual Spark UI.
FinOps cost estimate	Illustrative only	Replace assumptions with measured runtime, DBU usage, compute type and actual cost rate.
Full/Incremental integration	Pending separate task	Integrate Gold refresh once after each load's city-year loop and test safely.
GitHub submission	Repository exists	Commit final notebooks, report, data dictionary, and supporting documentation.

Conclusion

The supplied Databricks output supports that the Gold tables were created and passed the listed data-quality checks. The project can proceed with Power BI. Before treating this PDF as final submission evidence, insert the actual Spark UI DAG screenshot and replace illustrative FinOps values with measured compute usage and the applicable rate.

Source basis: Phase 2/Phase 3 project details and the Databricks run/validation outputs supplied in the conversation. Values and assumptions are explicitly identified above.